

Title: AI-Based Mini Project in .NET

BSCS-6A

Member:

- AYESHA NOOR [232201029]

Submitted to:

SIR UZAIR HASSAN



1. Project Introduction

1.1 Problem Statement

Modern users require quick, intelligent, and conversational answers to a wide variety of questions spanning writing assistance, programming help, language translation, and general knowledge. Traditional search engines return links requiring manual reading while static FAQ systems cannot adapt to dynamic queries. This project addresses the need for an intelligent conversational AI interface that understands natural language, remembers conversation context, supports multiple independent sessions, and provides user-specific history through secure authentication.

1.2 Objectives

- Develop a full-stack AI-powered chatbot using ASP.NET Core 8.0 and the MVC pattern
- Integrate the OpenRouter API (Mistral 7B free model) for AI-generated responses
- Implement user authentication (Login & Registration) using ASP.NET Core Identity
- Store and manage multiple independent chat sessions per logged-in user
- Build a professional dark-themed UI comparable to commercial AI chat applications
- Implement comprehensive input validation and error handling throughout the system
- Prepare a Dockerfile for containerized deployment of the application

2. System Design

2.1 Architecture Overview

The application follows the MVC (Model-View-Controller) architectural pattern within ASP.NET Core. Four distinct layers separate concerns: Presentation Layer (Razor Views + JavaScript), Controller Layer (ChatController & AccountController), Service Layer (GeminiService for API calls), and Data Layer (Entity Framework Core with SQLite).

Layer	Responsibility
Presentation	Razor Views (.cshtml) — Chat UI, Login page, Register page
Controller	ChatController (chat ops), AccountController (auth)
Service	GeminiService — builds OpenRouter API requests and parses responses
Data	ApplicationDbContext + SQLite — stores users via ASP.NET Core Identity
Model	ChatMessage, ChatSession, Auth ViewModels — shared data structures

2.2 Application Workflow

1. User visits the app — redirected to Login page (protected by [Authorize] attribute)
2. User registers a new account or signs in with existing credentials
3. ASP.NET Core Identity validates password hash and issues an authentication cookie

4. User lands on Chat page — a new session is automatically created in server Session
5. User types a message — sent via Fetch API (POST /Chat/Send) as JSON
6. ChatController validates input, loads session history, calls GeminiService
7. GeminiService POSTs full conversation history to OpenRouter API with Bearer token
8. Mistral 7B generates a response — returned as JSON to the service layer
9. Controller saves updated history to session store, returns JSON to browser
10. JavaScript removes typing indicator and renders the AI reply as a chat bubble
11. User can create new chats, switch sessions, or delete old ones from sidebar

2.3 Folder Structure

File / Folder	Purpose
Controllers/ChatController.cs	Chat operations: send, new session, switch, delete
Controllers/AccountController.cs	Register, login, logout via ASP.NET Core Identity
Services/GeminiService.cs	HTTP client for OpenRouter API
Models/ChatModels.cs	ChatMessage, ChatSession, GeminiRequest/Response
Models/AuthModels.cs	RegisterViewModel, LoginViewModel
Data/ApplicationDbContext.cs	EF Core context inheriting IdentityDbContext
Views/Chat/Index.cshtml	Main chat page with sidebar and JavaScript
Views/Account/Login.cshtml	Dark-themed login form
Views/Account/Register.cshtml	Registration form with validation
appsettings.json	API key, URL and AI model configuration
Program.cs	App startup — services, middleware, routing
Dockerfile	Multi-stage Docker build for deployment

3. Tools & Technologies

3.1 Development Environment

Tool	Version / Details
IDE	Microsoft Visual Studio 2022

.NET Framework	ASP.NET Core 8.0 (LTS)
Language	C# 12.0
Database	SQLite (file-based, zero configuration)
ORM	Entity Framework Core 8.0.11
Authentication	ASP.NET Core Identity 8.0.11

3.2 NuGet Packages

Package	Version
Microsoft.AspNetCore.Identity.EntityFrameworkCore	8.0.11
Microsoft.EntityFrameworkCore.Sqlite	8.0.11
Microsoft.EntityFrameworkCore.Tools	8.0.11
Microsoft.Extensions.Http	8.0.1
Microsoft.Extensions.Identity.Core	8.0.11

3.3 External API & Frontend

Service / Library	Details
OpenRouter API	openrouter.ai/api/v1/chat/completions — free AI gateway
AI Model	mistralai/mistral-7b-instruct:free — high quality, completely free
Font Awesome 6.4	Icon library via CDN
Google Fonts — Inter	Professional UI typeface
Vanilla JavaScript	ES6+ Fetch API for async backend communication

4. Code Snippets & Explanation

4.1 GeminiService.cs — OpenRouter API Call

This service sends the full conversation history to OpenRouter, enabling multi-turn context. The Authorization Bearer header is required by OpenRouter's API for authentication.

```
public async Task<string> GetReplyAsync(List<ChatMessage> history)
```

```

{
    var messages = history.Select(m => new {
        role = m.Role == "assistant" ? "assistant" : "user",
        content = m.Content
    }).ToList();
    var requestBody = new { model = _model, messages = messages };
    _httpClient.DefaultRequestHeaders.Add("Authorization", $"Bearer {_apiKey}");
    var response = await _httpClient.PostAsync(_apiUrl, httpContent);
}

```

4.2 ChatController.cs — Multi-Session Management

Each user has their own session namespace. Each chat gets a unique 8-character GUID. Session titles auto-update from the first message sent.

```

private string UserKey => User.Identity?.Name ?? "guest";
private string CreateNewSession()
{
    var sessionId = Guid.NewGuid().ToString("N")[..8];
    var sessions = GetAllSessions();
    sessions.Insert(0, new ChatSession { Id=sessionId, Title="New Chat" });
    SaveAllSessions(sessions);
    return sessionId;
}

```

4.3 AccountController.cs — User Registration

ASP.NET Core Identity handles password hashing automatically. On success the user is signed in immediately and redirected to the chat page.

```

var result = await _userManager.CreateAsync(user, model.Password);
if (result.Succeeded)
{
    await _signInManager.SignInAsync(user, isPersistent: false);
    return RedirectToAction("Index", "Chat");
}

```

4.4 Program.cs — Service Registration

Services use the built-in dependency injection container. ConfigureApplicationCookie sets the login redirect path for unauthenticated requests.

```

builder.Services.AddDbContext<ApplicationDbContext>(o =>
    o.UseSqlite("Data Source=chatbot.db"));
builder.Services.AddIdentity<IdentityUser, IdentityRole>(...)
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();
builder.Services.AddHttpClient<GeminiService>();

```

4.5 JavaScript — Async Message Send

Messages are sent without page reload using the Fetch API. A typing indicator with animated dots shows while waiting for the AI response.

```

const response = await fetch('/Chat/Send', {

```

```
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ message, sessionId: activeSessionId })
});
const data = await response.json();
if (!response.ok) showError(data.error);
else appendMessage('assistant', data.reply);
```

5. Application Preview

5.1 Login Page

The login page features a dark-themed card with the AI robot logo, email/password fields, Remember Me checkbox, and Sign In button. Users can navigate to the registration page via the 'Create one' link at the bottom.

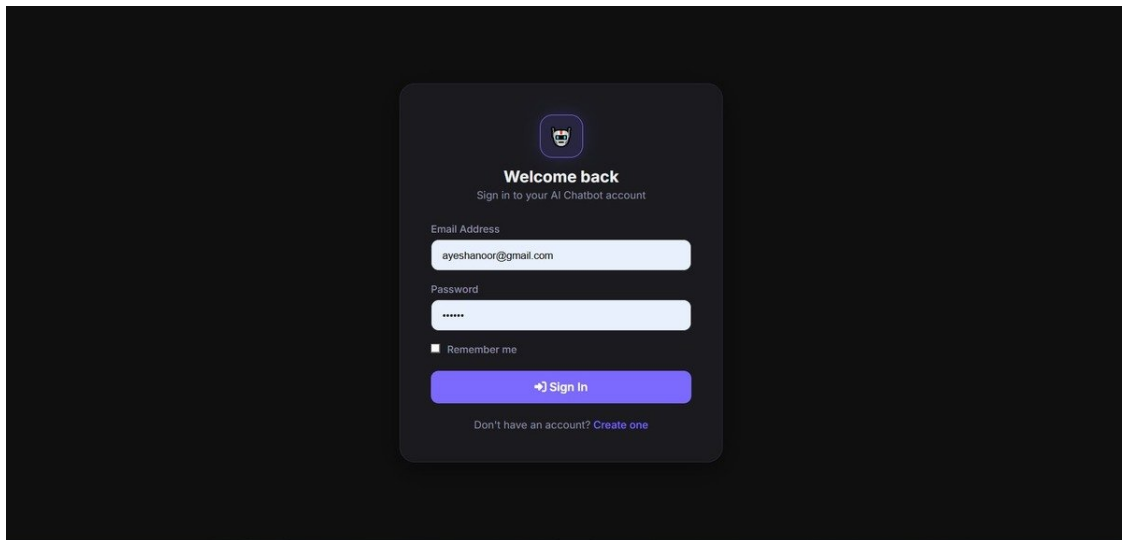


Figure 1: Login Page — Clean dark-themed authentication form

5.2 Login Error Handling

When invalid credentials are entered, the error message 'Invalid email or password.' appears in a red alert banner at the top of the form, demonstrating proper error handling.

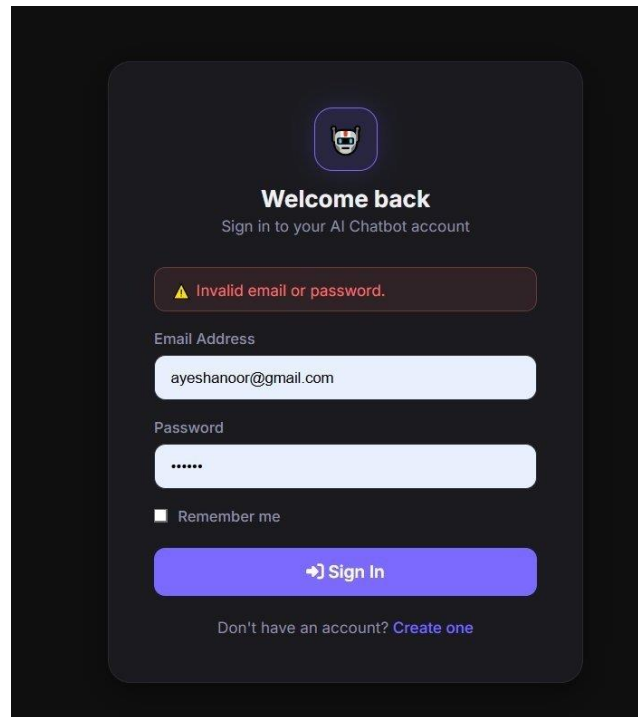


Figure 2: Login Error — Invalid credentials error message displayed

5.3 Registration Page

New users register with their full name, email, password (min. 6 characters), and password confirmation. ASP.NET Core Identity validates the data server-side and displays errors inline.

A dark-themed registration page for 'AI Chatbot'. At the top is a small AI logo. Below it, the text 'Create account' is displayed in a bold, light blue font, followed by 'Join AI Chatbot for free' in a smaller, lighter blue font. The form contains four input fields: 'Full Name' with 'John Doe', 'Email Address' with 'you@example.com', 'Password' with 'Min. 6 characters', and 'Confirm Password' with 'Repeat password'. A large, rounded blue button with a white user icon and the text 'Create Account' is centered below the inputs. At the bottom, a link says 'Already have an account? Sign in'.

Figure 3: Registration Page — Create Account form with validation

5.4 Chat Welcome Screen

After login, the user lands on the chat interface. New sessions show a welcome screen with the AI logo, greeting, and four suggestion chips. The sidebar shows all sessions and the user's email at the bottom-left.

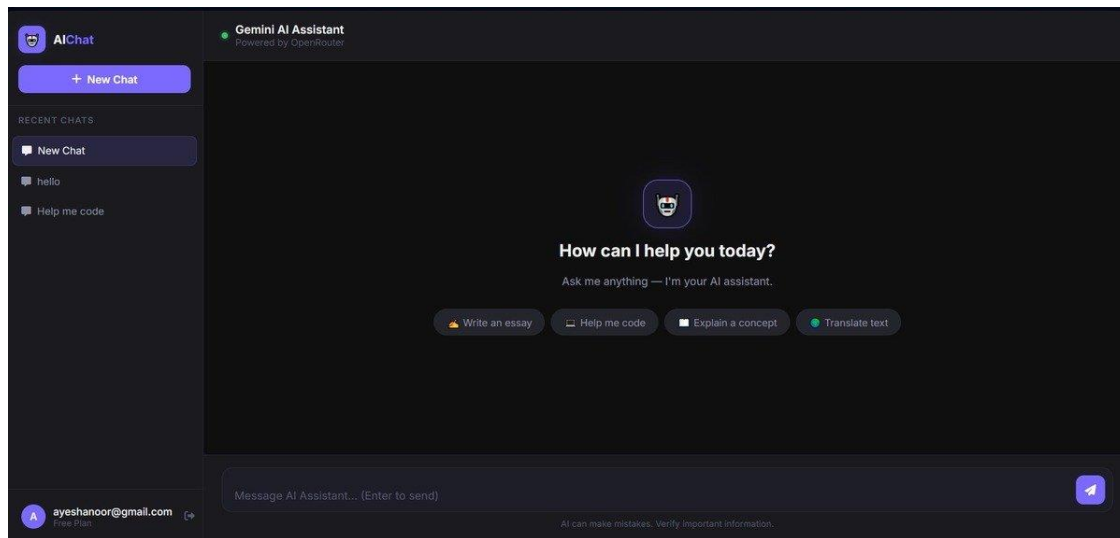


Figure 4: Welcome Screen — New chat session with suggestion chips and session sidebar

5.5 Chat Conversation — Code Assistance

The user clicked 'Help me code'. User messages appear as purple gradient bubbles on the right. AI responses appear in dark bubbles on the left. The session is titled automatically from the first message.

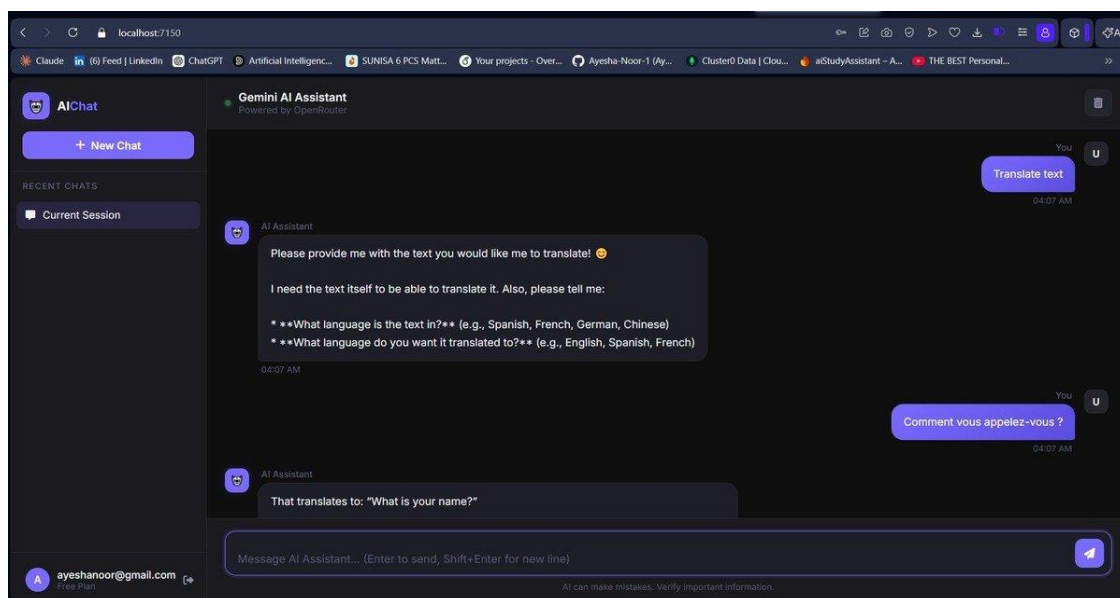


Figure 5: Code Assistance — Multi-turn conversation with message bubbles and session sidebar

5.6 Chat Conversation — Language Translation

This session shows the translation feature. The user sent a French phrase and the AI correctly translated it to English, demonstrating multi-turn context awareness across the full conversation history.

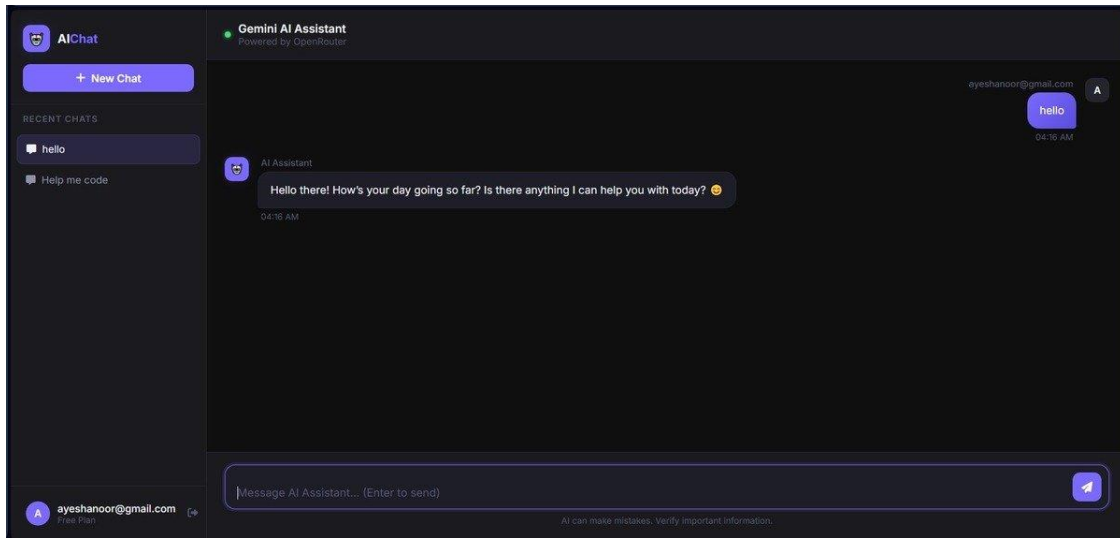


Figure 6: Translation — French to English translation with full context awareness

5.7 Multiple Chat Sessions

The sidebar shows three separate sessions: 'hello', 'Help me code', and 'New Chat'. Each session is stored independently. Clicking any session instantly loads its full conversation history without a page reload.

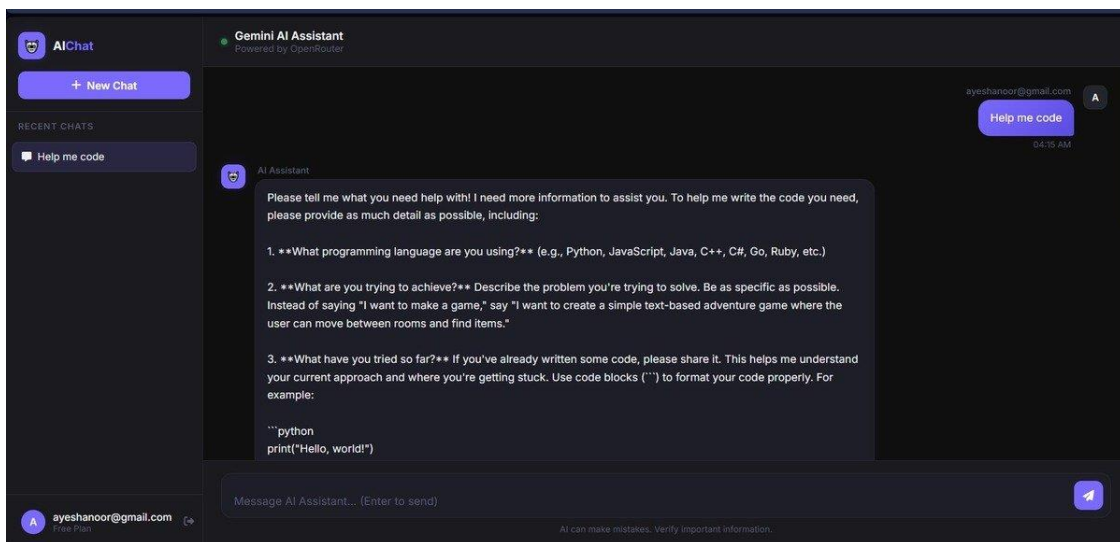


Figure 7: Multi-Session — Sidebar showing multiple independent chat sessions

5.8 UI Feature Summary

Feature	Description
Dark Theme	Professional dark color scheme with CSS custom properties
Message Bubbles	Purple gradient for user, dark card for AI responses
Avatars	User initial letter (right), robot emoji for AI (left)

Typing Indicator	Three animated bouncing purple dots during AI generation
Session Sidebar	Lists all conversations with auto-generated titles from first message
Welcome Screen	Shown on empty sessions with 4 clickable suggestion chips
Auto-Resize Input	Textarea grows up to 120px as user types
Status Indicator	Animated pulsing green dot showing AI is online
Logout Button	Arrow icon in sidebar footer, submits secure logout form

6. Testing

6.1 Functional Test Cases

Test Case	Result
Register with valid data	✓ Account created, redirected to chat
Register — passwords don't match	✓ Error: 'Passwords do not match'
Register — password under 6 chars	✓ Error: 'Password must be at least 6 characters'
Login with correct credentials	✓ Auth cookie set, redirected to /Chat
Login with wrong password	✓ Error: 'Invalid email or password.' (Figure 2)
Send empty message	✓ 400 returned, error shown without API call
Send message over 1000 characters	✓ 400 returned, 'Message too long' error shown
Send valid chat message	✓ AI response displayed with typing animation
Click suggestion chip	✓ Chip text auto-fills input and sends
Create new chat	✓ New session, welcome screen shown, old chat preserved
Switch between sessions	✓ Full message history loads instantly

Delete a session	✓ Removed from sidebar, switches to next session
Logout	✓ Cookie cleared, redirected to login
Access /Chat without login	✓ Auto-redirected to /Account/Login

6.2 Sample Inputs & Outputs

User Input	AI Response (Mistral 7B)
hello	Hello there! How's your day going? Is there anything I can help you with? 😊
Help me code	Please tell me what you need help with! What language are you using? What are you trying to achieve?
Comment vous appelez-vous?	That translates to: 'What is your name?'
What is 2 + 2?	2 + 2 = 4
Write a haiku about coding	Fingers on keyboard / Logic flows through silent wires / Bug found, peace restored

7. Results & Discussion

7.1 What Worked Well

- **ASP.NET Core Identity:** Seamlessly handled registration, password hashing, and cookies with minimal custom code
- **OpenRouter API:** Mistral 7B provided accurate, context-aware responses by sending full conversation history
- **Session-Based History:** Built-in session middleware stored per-user, per-session data effectively
- **Multi-Session Management:** Users maintain multiple independent conversations, switch and delete without page reloads
- **Professional UI:** Dark theme with animations, typing indicator, and sidebar delivered a polished UX
- **Error Handling:** API errors (429, 500), network failures, and empty inputs all handled gracefully

7.2 Limitations

- **Session Storage:** Chat history stored in memory — lost on server restart. Needs database persistence for production
- **API Rate Limits:** Free tier has per-minute limits. Heavy testing triggers temporary 429 errors
- **No Markdown Rendering:** AI responses with markdown (code blocks, bold) display as raw text
- **No Image Upload:** Current version does not support image input or visual analysis

8. Future Improvements

- **Persistent Database Storage:** Save chat sessions to SQLite tables so history survives restarts
 - **Markdown Rendering:** Integrate Marked.js to render code blocks and formatting in AI responses
 - **Image Upload & Vision:** Support image input using OpenRouter vision-capable models
 - **Persona Mode Selector:** Teacher, Coder, Doctor modes using custom system prompts
 - **Streaming Responses:** Server-Sent Events for real-time token-by-token streaming
 - **Mobile Responsive Design:** Collapsible sidebar and touch-optimized input for mobile devices
-

9. Docker Configuration

9.1 Overview

Although Docker was not used to run the application during development (the app runs directly via Visual Studio), a production-ready Dockerfile has been prepared. Docker containerization allows deployment to any server or cloud platform without manually installing .NET 8.0 or managing dependencies. The Dockerfile uses a multi-stage build — a Docker best practice that produces a small, secure image by separating the build stage from the runtime stage.

9.2 Complete Dockerfile

```
# STAGE 1: BUILD — uses full .NET 8 SDK (~700MB)
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app
COPY *.csproj ./
RUN dotnet restore
COPY . ./
RUN dotnet publish -c Release -o out

# STAGE 2: RUNTIME — uses smaller ASP.NET image (~200MB)
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/out .
EXPOSE 80
ENTRYPOINT ["dotnet", "AIChatbotApp.dll"]
```

9.4 Build & Run Commands

Step 1 — Build image:

```
docker build -t aichatbotapp .
```

Step 2 — Run container:

```
docker run -p 8080:80 aichatbotapp
```

Step 3 — Open in browser:

`http://localhost:8080`

The `-p 8080:80` flag maps port 80 inside the container to port 8080 on the host. The multi-stage build produces a ~200MB image instead of ~700MB by excluding the SDK and source code from the final image.

10. Conclusion

This project successfully demonstrates the development of a complete, production-quality AI-powered chatbot web application using ASP.NET Core 8.0. All core features are fully functional: user registration and login with error handling, a professional dark-themed chat interface, multi-turn AI conversations powered by the OpenRouter Mistral 7B model, and multi-session management with independent conversation histories per user.

The screenshots in Section 5 confirm the application works as designed — from the login/register pages (Figures 1–3) through the chat welcome screen (Figure 4), code assistance and translation conversations (Figures 5–6), to multi-session sidebar management (Figure 7). The Docker configuration in Section 9 ensures the application can be consistently deployed to any environment.

This project provided practical experience in AI API integration in C#, ASP.NET Core Identity, Entity Framework Core with SQLite, server-side session management, asynchronous JavaScript Fetch API, professional CSS dark theming, and Docker multi-stage containerization.
